Bus Sensor Data Pipeline — Complete Setup Log

Project: Bus Censor Data Pipeline
Date: 2026-08-05
System: Arch Linux + Hyprland, i7-13700H, RTX 4050
User: void-god

Architecture

```
Oracle XE (Docker) → Airflow 3.3.0 → S3 (Parquet) → Snowpipe → Snowflake → Superset
         ↑                ↑                ↑
    1000 fake rows    Daily schedule   Hive partitioning
                      (port 8081)
```

Phase 1: Environment Setup

1.1 Docker Containers Already Running

Container	Image	Port	Purpose
oracle	gvenzl/ oracle-xe:21-slim	1521	Source database
portainer	portainer/portainer-ce	9000	Docker management
odysseus	odysseus-odysseus	7000	AI assistant
searxng	searxng/searxng	8080	Search engine
chromadb	chromadb/chroma	8100	Vector DB
ntfy	binwiederhier/ntfy	8091	Notifications

1.2 AWS S3 Bucket

```
# Bucket created: mkp-s3-data in ap-south-1
# AWS CLI configured with credentials
aws s3 ls s3://mkp-s3-data/
```

1.3 Python Environment (pipx)

```
# Airflow installed via pipx (isolated from system Python)
pipx install apache-airflow

# Packages injected into Airflow venv
pipx inject apache-airflow boto3
pipx inject apache-airflow setuptools
pipx inject apache-airflow oracledb       # cx_Oracle failed on Python 3.14
pipx inject apache-airflow pandas
pipx inject apache-airflow pyarrow
pipx inject apache-airflow apache-airflow-providers-amazon
```

Important: Verify packages using the venv's Python, NOT `pipx run --spec`:

```
~/.local/share/pipx/venvs/apache-airflow/bin/python -c "import pandas; print(pandas.__version__)"
~/.local/share/pipx/venvs/apache-airflow/bin/python -c "import oracledb; print(oracledb.__version__)"
```

Phase 2: Airflow 3.x Setup (Major Differences from 2.x)

2.1 PATH Fix

```
# pipx exposed the binary as 'apache-airflow', not 'airflow'
ln -s ~/.local/share/pipx/venvs/apache-airflow/bin/airflow ~/.local/bin/airflow
airflow version   # 3.3.0
```

2.2 Database Initialization

```
# Airflow 2.x: airflow db init
# Airflow 3.x: airflow db migrate
export AIRFLOW_HOME=~/airflow
airflow db migrate
```

2.3 Port Conflict Fix (SearXNG on 8080)

```
# Airflow tried to bind to 8080 but SearXNG already owns it
# Permanently changed port in config:
sed -i '1454s/port = 8080/port = 8081/' ~/airflow/airflow.cfg
grep "^port = 8081" ~/airflow/airflow.cfg   # verify
```

2.4 Start Airflow (Standalone Mode)

```
# Airflow 3.x 'standalone' replaces webserver + scheduler + triggerer
export AIRFLOW_HOME=~/airflow
nohup airflow standalone > ~/airflow/standalone.log 2>&1 &

# Check it's running
ss -tlnp | grep 8081

# Stop Airflow
pkill -9 -f "airflow"
```

2.5 Login

- Password stored in: `~/airflow/simple_auth_manager_passwords.json.generated`
- URL: http://localhost:8081
- Username: `admin`
- Password: read from JSON file above

Phase 3: Oracle Schema

3.1 Create Table + 1000 Fake Rows

```
docker start oracle   # if not running
docker exec -i oracle sqlplus SYSTEM/admin@XE << 'EOF'
CREATE TABLE bus_sensors (
    sensor_id NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    bus_id VARCHAR2(20) NOT NULL,
    sensor_type VARCHAR2(50),
    reading_value NUMBER(10,4),
    unit VARCHAR2(20),
    latitude NUMBER(10,6),
    longitude NUMBER(10,6),
    recorded_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    status VARCHAR2(20) DEFAULT 'ACTIVE'
);

INSERT INTO bus_sensors (bus_id, sensor_type, reading_value, unit, latitude, longitude, status)
SELECT
    'BUS-' || LPAD(ROWNUM, 3, '0'),
    CASE MOD(ROWNUM, 5)
        WHEN 0 THEN 'SPEED'
        WHEN 1 THEN 'TEMPERATURE'
        WHEN 2 THEN 'VIBRATION'
        WHEN 3 THEN 'FUEL_LEVEL'
        ELSE 'ENGINE_RPM'
    END,
    ROUND(DBMS_RANDOM.VALUE(10, 120), 2),
    CASE MOD(ROWNUM, 5)
        WHEN 0 THEN 'KM/H'
        WHEN 1 THEN 'CELSIUS'
        WHEN 2 THEN 'G_FORCE'
        WHEN 3 THEN 'PERCENT'
        ELSE 'RPM'
    END,
    ROUND(DBMS_RANDOM.VALUE(12.90, 13.10), 6),
    ROUND(DBMS_RANDOM.VALUE(77.50, 77.70), 6),
    CASE WHEN DBMS_RANDOM.VALUE(0,1) > 0.9 THEN 'WARNING' ELSE 'ACTIVE' END
FROM dual
CONNECT BY ROWNUM <= 1000;

COMMIT;
EXIT;
EOF
```

3.2 Verify

```
docker exec -i oracle sqlplus -S SYSTEM/admin@XE <<< "SELECT COUNT(*) FROM bus_sensors;"
# Output: 1000
```

Phase 4: Project Structure

```
/home/void-god/Ajay Mkp/A projects/Learning/Snow_flake/projects_&_assessments/bus-censor-project/
├── dags/
│   └── bus_censor_oracle_to_s3.py          ← Real file (git tracked)
├── sql/
│   ├── oracle/
│   │   └── create_tables.sql
│   └── snowflake/
│       ├── 01_storage_integration.sql
│       ├── 02_external_stage.sql
│       ├── 03_snowpipe.sql
│       ├── 04_raw_tables.sql
│       ├── 05_clean_transforms.sql
│       └── 06_serve_views.sql
├── dbt/models/
├── data/
└── docs/
    └── README.md

~/airflow/
├── dags/
│   └── bus_censor_oracle_to_s3.py          ← Symlink to project file
├── logs/
├── plugins/
└── airflow.db
```

Symlink (for Airflow to see the DAG)

```
ln -s "/home/void-god/Ajay Mkp/A projects/Learning/Snow_flake/projects_&_assessments/
↪  bus-censor-project/dags/bus_censor_oracle_to_s3.py"   ~/airflow/dags/bus_censor_oracle_to_s3.py
```

Phase 5: The DAG (Airflow 3.x Compatible)

File: `bus-censor-project/dags/bus_censor_oracle_to_s3.py`

```
from datetime import datetime, timedelta
from airflow import DAG
from airflow.providers.standard.operators.python import PythonOperator
from airflow.providers.amazon.aws.hooks.s3 import S3Hook
import pandas as pd
import oracledb
import io

ORACLE_HOST, ORACLE_PORT, ORACLE_USER, ORACLE_PASS, ORACLE_SERVICE = "localhost", 1521, "SYSTEM",
↪  "admin", "XE"
S3_BUCKET, S3_PREFIX = "mkp-s3-data", "bus-censor/raw"

default_args = {
    "owner": "void-god",
```

```
    "depends_on_past": False,
    "email_on_failure": False,
    "email_on_retry": False,
    "retries": 1,
    "retry_delay": timedelta(minutes=5),
}

def extract_oracle_to_s3(**context):
    execution_date = context["ds"]
    conn = oracledb.connect(user=ORACLE_USER, password=ORACLE_PASS,
                            dsn=oracledb.makedsn(ORACLE_HOST, ORACLE_PORT, service_name=ORACLE_SERVICE))
    df = pd.read_sql(f"SELECT * FROM bus_sensors WHERE TRUNC(recorded_at) <= TO_DATE('{execution_date}',
     ↪  'YYYY-MM-DD')", conn)
    conn.close()
    if df.empty:
        return f"s3://{S3_BUCKET}/{S3_PREFIX}/{execution_date}/no_data.flag"

    # CRITICAL FIX: Oracle returns UPPERCASE column names
    df.columns = [c.lower() for c in df.columns]
    df["recorded_at"] = pd.to_datetime(df["recorded_at"])

    buffer = io.BytesIO()
    df.to_parquet(buffer, index=False, engine="pyarrow")
    buffer.seek(0)
    s3_key = f"{S3_PREFIX}/year={execution_date[:4]}/month={execution_date[5:7]}/
     ↪  day={execution_date[8:10]}/bus_sensors_{execution_date}.parquet"
    S3Hook(aws_conn_id="aws_default").load_bytes(bytes_data=buffer.getvalue(), key=s3_key,
     ↪  bucket_name=S3_BUCKET, replace=True)
    print(f"Uploaded {len(df)} rows to s3://{S3_BUCKET}/{s3_key}")
    return f"s3://{S3_BUCKET}/{s3_key}"

with DAG("bus_censor_oracle_to_s3", default_args=default_args,
         description="Oracle to S3 Parquet", schedule="@daily",
         start_date=datetime(2026, 7, 30), catchup=False,
         tags=["bus","oracle","s3"]) as dag:
    PythonOperator(task_id="extract_oracle_to_s3", python_callable=extract_oracle_to_s3)
```

Key Airflow 3.x Changes from 2.x

Feature	2.x	3.x
Init DB	`airflow db init`	`airflow db migrate`
Schedule	`schedule_interval="@daily"`	`schedule="@daily"`
PythonOperator import	`airflow.operators.python`	`airflow.providers.standard.operators.pyt`
Start services	`webserver` + `scheduler` (2 terminals)	`airflow standalone` (1 command)
User creation	`airflow users create`	Auto-created by standalone
Trigger DAG	`airflow dags trigger <id>`	Same
Test task	`airflow tasks test <dag> <task> <date>`	Same

Phase 6: Problems & Fixes

Problem 1: `airflow` command not found

Cause: pipx exposed binary as `apache-airflow`, not `airflow`
Fix: `ln -s ~/.local/share/pipx/venvs/apache-airflow/bin/airflow ~/.local/bin/airflow`

Problem 2: `cx_Oracle` build failure

Cause: C extensions incompatible with Python 3.14 on Arch
Fix: Used `oracledb` (Oracle's thin driver, no client needed)

Problem 3: `pipx run --spec` verification fails

Cause: `pipx run --spec` treats "python" as a package name, not the venv interpreter
Fix: Use `~/.local/share/pipx/venvs/apache-airflow/bin/python -c "import ..."`

Problem 4: Airflow 3.x CLI commands don't exist

Cause: Most online docs are for Airflow 2.x; 3.x changed everything
Fix: Use `airflow db migrate`, `airflow standalone`, `airflow tasks test`

Problem 5: Port 8080 conflict with SearXNG

Cause: Both Airflow and SearXNG default to 8080
Fix: Changed `~/airflow/airflow.cfg` line 1454: `port = 8081`

Problem 6: DAG not appearing in UI

Cause: `schedule_interval` is invalid in Airflow 3.x
Fix: Changed to `schedule="@daily"`

Problem 7: Task stuck in "queued" for 20+ minutes

Cause: DAG was **paused** — unpause required before runs execute
Fix: Click toggle in UI or `airflow dags unpause bus_censor_oracle_to_s3`

Problem 8: `KeyError: 'recorded_at'`

Cause: Oracle returns column names as UPPERCASE (`RECORDED_AT`)
Fix: `df.columns = [c.lower() for c in df.columns]`

Problem 9: S3Hook auth warning

Cause: No `aws_default` Airflow connection configured
Fix: boto3 automatically falls back to `~/.aws/credentials` (no action needed)

Phase 7: Verification Commands

Test the full pipeline via CLI

```
airflow tasks test bus_censor_oracle_to_s3 extract_oracle_to_s3 2026-08-05
```

Verify S3 output

```
aws s3 ls s3://mkp-s3-data/bus-censor/raw/ --recursive
```

Expected output

```
2026-08-05 12:46:25  40878 bus-censor/raw/year=2026/month=08/day=05/bus_sensors_test.parquet
2026-08-05 13:00:45  40878 bus-censor/raw/year=2026/month=08/day=05/bus_sensors_2026-08-05.parquet
```

Current Status

Component	Status
Oracle XE (1000 rows)	✅ Running
Airflow 3.3.0	✅ On port 8081
DAG parsed	✅ No import errors
Task execution	✅ Via `airflow tasks test`
Parquet → S3	✅ 40.9 KB file uploaded
Hive partitioning	✅ `year=2026/month=08/day=05/`
AWS credentials	✅ Via `~/.aws/credentials`

Next Steps (Snowflake Phase)

1. **Storage Integration** — Allow Snowflake to read S3
2. **External Stage** — Point to `s3://mkp-s3-data/bus-censor/raw/`
3. **Snowpipe** — Auto-ingest new Parquet files
4. **Raw Tables** — `RAW_BUS_SENSORS` landing zone
5. **Clean Transforms** — Typed columns, deduplicated
6. **Serve Views** — Ready for Apache Superset dashboards

Daily Workflow

```
# Start everything
docker start oracle
nohup airflow standalone > ~/airflow/standalone.log 2>&1 &

# Check status
ss -tlnp | grep 8081
```

```
# Stop everything
pkill -9 -f "airflow"
```

File Locations

File	Path
DAG source	`/home/void-god/Ajay Mkp/A projects/Learning/Snow_flake/projects_&_assessments/bus-censor-project/dags/bus_censor_oracle_to_s3.py`
DAG symlink	`~/airflow/dags/bus_censor_oracle_to_s3.py`
Airflow config	`~/airflow/airflow.cfg`
Airflow DB	`~/airflow/airflow.db`
Standalone log	`~/airflow/standalone.log`
Admin password	`~/airflow/simple_auth_manager_passwords.json.generated`
Oracle data	Docker volume `oracle`
S3 bucket	`s3://mkp-s3-data/bus-censor/raw/`